COMPUTER SCIENCE + COMPUTER ENGINEERING

06

Module 24: Engine architecture, simulation and resource ownership

Expanded Volume 05 / four-lesson module

4 lessons · 20 lesson questions · 5 review questions · Integrating project

PACKAGE 06 / EXPANDED VOLUME 05 / SEPTEMBER 28, 2026
Self-study: derive, implement, test and explain.

Module 24 — Engine architecture, simulation and resource ownership

Prerequisites: Modules 12, 17-18, 21-23.

Planning allocation: 80-120 planning hours, including the project's work and review.

Lesson	Outcome
24.1 — Fixed updates, interpolation and overload policy	Separate simulation time from rendering time and real elapsed time.
24.2 — Entities, components, handles and data-oriented storage	Design identity separately from memory location.
24.3 — Allocators, task graphs and frame-resource lifetime	Make temporary storage and parallel dependencies observable.
24.4 — Subsystem boundaries, save formats and reloadability	Choose interfaces that make change and failure manageable.

Original module project

Build a small engine with a fixed-step world, generational entity handles, explicit resource ownership and an inspectable task graph. Keep the game rules separate enough to run automated headless tests.

Integrating milestones

1. P24-A: fixed ticks, interpolation and explicit overload accounting.
2. P24-B: generational identity and deferred structural edits.
3. P24-C: checked arena plus serial/parallel task-graph validation.
4. P24-D: schema migration, transactional loads and safe reload/shutdown.

Evidence to retain

- Tick-indexed input log; time invariant tests; entity mutation tests.
- Task trace and critical path; arena guard tests; save fixtures; stale reload and initialization-failure logs.

Read the four lessons, keep the assigned readings, implement the workbook labs and use the module review before advancing. The reference code is a diagnostic aid; milestone completion is learner work.

LESSON 24.1 / MODULE 24

Fixed updates, interpolation and overload policy

> **Learning outcome:** Separate simulation time from rendering time and real elapsed time.

Simulation time is a design choice

A rendering callback arrives according to scheduling, device load and presentation constraints. Game rules should not silently inherit that variable interval. Maintain a fixed simulation step h and an accumulator of accepted elapsed time. Add elapsed time, run complete steps while enough time remains, and retain the fractional remainder for rendering. The simulation advances in whole ticks, while rendering can interpolate between saved states. [R27]

Keep previous and current simulation snapshots. After completing available steps, alpha=accumulator/h should lie in [0,1) under the selected policy. Rendering lerp(previous,current,alpha) intentionally displays a state between two completed ticks, generally about one tick behind the latest real-time trajectory. It avoids predicting an uncomputed future state but introduces that interpolation delay. Camera or user-interface elements may need their own latency policy.

Inputs belong to ticks

Sample or receive input events with enough timing information to assign them consistently to simulation ticks. Do not apply the same press repeatedly merely because multiple catch-up ticks occur. Separate continuous state, such as a held movement direction, from discrete edges, such as a one-time jump press. Record tick-indexed input when building replay tests.

Fixed h is not a universal determinism switch. Random-number order, floating-point behavior, unordered iteration, task races, input assignment and external I/O can all change results. State the reproducibility level actually supported: same executable and input stream, same platform family, or cross-platform bit identity. A successful same-process test does not prove the strongest claim.

Overload needs an explicit response

An application that always tries to simulate all accumulated time can fall behind: catch-up work causes a longer frame, which creates still more catch-up work. Choose a policy. A teaching interactive policy clamps an extreme incoming frame interval, performs at most K steps, then discards whole remaining ticks while retaining a sub-tick remainder. Record every discarded interval. This sacrifices synchronization with real elapsed time to protect responsiveness; it must not be used silently in a network-authoritative simulation or scientific model.

Alternatives include slowing presentation, reducing optional work, pausing with a warning or running a headless simulation without dropping ticks. The correct policy depends on product requirements. A cap of eight steps is not automatically better than four; it changes the maximum compute burst and tolerated latency.

Measure three clocks

Report real elapsed time, accepted simulation time and dropped time separately. The invariant for the simple drop policy is old_accumulator+elapsed = steps*h + new_accumulator + dropped, with all terms in the same units. Use integer time units in small tests so fractional-millisecond arithmetic does not obscure the policy. The C++ reference uses a checked integer-microsecond budget helper.

Test normal cadence, a 100-ms stall, repeated stalls, zero elapsed time and invalid negative input. Then verify a world update with constant velocity. Keep a render interpolation test separate from the simulation state test; comparing rendered positions at different callback times is not the same as comparing states after the same tick count.

Worked example

With h=10 ms and accumulated time 25 ms, execute two steps and retain 5 ms. The interpolation fraction is 5/10=0.5, matching the original quiz. If previous x=1.0 and current x=1.2, render x=1.1.

Under a maximum of four steps, a 105-ms accumulator executes 40 ms of simulation. The 65-ms remainder contains six whole 10-ms ticks and 5 ms. The teaching drop policy records 60 ms dropped and retains 5 ms, so alpha=0.5. The accounting is 105=40+60+5.

For velocity 3 units/s and h=0.01 s, each tick adds 0.03 units. One hundred ticks advance 3 units under this simple integration rule. Ten calls carrying 100 ms each can produce a different tick count when the overload cap discards time, so do not expect the same result without considering the policy.

Original lab assignment

Implement an accumulator loop and inject long frames. Pass: simulation advances consistently under normal load, interpolation is bounded, and the overload behavior is tested rather than silently spiraling.

Assigned study

R27: Fix Your Timestep!; R20: game loops and timing. Use the article as a starting model, not a guarantee of determinism.

Focused study: R27: accumulator and overload model. R20: engine timing. Treat the worked policy as an explicit design choice, not universal advice.

Check your understanding

1. What fraction remains after two 10 ms steps from 25 ms?
2. Does a fixed timestep alone guarantee determinism?
3. Why must catch-up work be bounded or otherwise controlled?
4. Under the teaching drop policy, h=10 ms, max steps=4 and accumulated time=105 ms. Give steps, remainder, dropped time and alpha.
5. Why can two runs using the same fixed h still diverge? Give two causes other than h.

LESSON 24.2 / MODULE 24

Entities, components, handles and data-oriented storage

> **Learning outcome:** Design identity separately from memory location.

Identity must survive storage changes

An entity is an identity in the simulated world. Its memory address is one possible temporary location, not its durable name. A vector can reallocate, a dense component store can move its last element into a removed slot, and an allocator can reuse a freed address for a different object. A pointer that still has plausible bits may therefore refer to the wrong lifetime.

Use a handle consisting of slot index and generation. A live slot stores its current generation; lookup succeeds only if the index is in range, the slot is alive and generations match. On destruction, invalidate the slot and advance its generation before reuse. A stale handle then fails instead of silently referring to the replacement entity. A finite generation counter can wrap, so choose a sufficiently wide counter and a stated wrap policy, such as permanently retiring an exhausted slot. [R11]

Components organize data around access

Begin with position and velocity components. An array-of-structures representation groups one entity's fields; a structure-of-arrays representation groups the same field across entities. The best choice depends on the hot queries and mutation patterns. Updating every position from every velocity may favor contiguous field arrays, but frequent heterogeneous operations can change the tradeoff. Measure a relevant workload rather than adopting data-oriented terminology as proof of speed.

A sparse-set design commonly maps entity slots to dense component indices and maps dense indices back to entities. Removal can swap the last dense element into a hole and update the moved entity's reverse mapping. This is efficient but makes borrowed references short-lived. A handle lookup at a synchronization boundary is safer than keeping an unchecked component pointer across structural changes.

Iteration and mutation need a transaction boundary

Destroying or creating entities while traversing a dense store can invalidate iteration and skip or repeat items. Queue structural commands during the update, then apply them at a defined boundary. Settle conflicting commands deterministically: a queued destroy followed by a write to the same handle should reject or follow a documented policy. Do not let vector iteration order determine the outcome accidentally.

Deferred destruction does not mean an entity is safe for all uses until memory is freed. The world must define whether a pending-destroy entity participates in later systems within the same tick. GPU resource retirement is another layer: removing an entity does not free a mesh still referenced by in-flight draws.

Durable IDs and runtime handles are different

A save file may use stable scene IDs and translate them to runtime handles when loaded. Saving only slot index and generation couples the format to a transient allocation history. Preserve relationships using the durable identity scheme, then resolve them transactionally after creating the new world.

The required evidence includes stale-handle tests, duplicate destruction, invalid index, slot reuse and mutation during iteration. Benchmarks should include creation, destruction and common queries, with input

size and iteration order stated. A faster store that accepts stale handles has failed the problem it was meant to solve.

Worked example

Handle (7,3) references slot 7 at generation 3. Destroy the entity, increment that slot to generation 4 and reuse it. Lookup of (7,3) must fail; lookup of the new (7,4) succeeds. Comparing only index 7 would confuse two different lifetimes.

In a dense position array, entity A is at index 0, B at 1 and C at 2. Remove B by moving C into index 1. Update C's sparse mapping to 1, shorten the dense array, and invalidate B's mapping. A borrowed pointer to index 1 now names C, demonstrating why address-based identity is unsafe.

During a tick, queue destroy(B) while iterating A,B,C. Complete the selected iteration policy, then apply the command once at the barrier. Test that C was not accidentally skipped by an immediate erase and that a repeated destroy(B) is rejected or treated idempotently according to the documented API.

Original lab assignment

Implement a small generational entity store with two components and deferred destruction. Pass: stale handles fail safely, iteration survives queued changes, and benchmarks include creation, deletion and common queries.

Assigned study

R20: game-state organization; R26: gameplay architecture for comparison; R11: lifetime and interface guidance.

Focused study: R11: lifetime, ownership and interfaces. R20: game-state organization. R26: compare, rather than replace, the small implementation.

Check your understanding

1. Why separate identity from an object address?
2. Is handle (7,3) valid for a slot now at generation 4?
3. Why defer structural changes during a component iteration?
4. After swap-removing dense element B and moving C into its slot, which mapping must change?
5. Why should a durable scene ID normally be separate from a runtime slot/generation handle?

LESSON 24.3 / MODULE 24

Allocators, task graphs and frame-resource lifetime

> **Learning outcome:** Make temporary storage and parallel dependencies observable.

Temporary memory is a lifetime claim

A linear arena allocates by advancing an offset through a fixed block. Resetting the offset is cheap only because the program promises that no remaining consumer will use any old allocation. That promise includes worker tasks, callbacks and GPU work, not just the function that allocated the memory. An arena's convenience does not exempt it from alignment, bounds or object-lifetime rules.

For size n and alignment a, round the current offset up to an aligned position before checking that n fits. Use overflow-safe arithmetic. A C++ byte arena that returns raw storage does not automatically construct objects or call destructors. The small exercise allocates byte ranges only; extending it to nontrivial C++ objects requires explicit lifetime management. [R11]

A task graph exposes ordering

A task is work with declared inputs, outputs and dependencies. A directed acyclic graph permits a task only after its predecessors have produced the required outputs. First implement a serial topological executor with cycle detection and stable tie order. This provides a reference execution order and makes missing dependencies easier to reproduce before adding scheduling nondeterminism.

Edges alone are not a complete race proof. If two tasks write the same array but neither depends on the other, a graph scheduler may legally run them together while the program races. Declare resource access or partition ownership so the graph can be checked against the actual data. Separate a logical task graph from the scheduling strategy that maps ready tasks to worker threads.

The critical path bounds improvement

If A and B can run independently and C consumes both, ideal parallel time is at least max(time(A),time(B)) +time(C), not the sum divided by worker count. More workers do not shorten a serial dependency chain. Scheduling overhead, cache behavior, contention and load imbalance can increase actual time beyond that lower bound.

Use a trace with start, finish, worker, dependencies and named resources. A bar chart is less useful when tasks are labeled only job1 and job2. Compare a serial run and a parallel run on identical inputs and validate results before analyzing speedup. A parallel task graph that produces wrong output has no meaningful performance win.

Frame ownership joins CPU and GPU lifetimes

A frame arena can contain CPU draw packets, while a staging arena may contain bytes needed for GPU upload. Those may have different retirement points. The CPU packet arena can reset after all CPU consumers copy what they need; GPU-visible staging storage may require submission completion. Grouping both under the name frame memory does not make their lifetimes equal.

Use a live-use token or completion serial in the reference model. Refuse reset while any use is outstanding. In a multiqueue or multithreaded engine, the token must actually represent every consumer. A counter modified without synchronization merely replaces one lifetime bug with a race.

Make failure tests constructive

Submit a task that retains a range, then attempt reset; the checked model must reject it. Add a dependency cycle and verify an actionable error. Remove a required edge in a scratch copy and demonstrate the discrepancy with the serial oracle. Record the limitation that the packaged scheduler model is serial; a race-free parallel executor is part of the full lab, not proven by a topological-sort test.

Worked example

An arena has capacity 64 bytes and offset 17. A request for 8 bytes aligned to 16 begins at 32 and ends at 40. The padding is 15 bytes, and the next offset is 40. A request that would end above 64 must fail without changing the previous offset.

Tasks A=2 ms and B=5 ms are independent; C=3 ms requires both. Serial time is 10 ms. Ideal unlimited-worker time is at least 5+3=8 ms, for a maximum speedup of 1.25 under this model. Scheduling overhead can make the actual speedup smaller.

If a frame arena's last use is submission 42 and the completed serial is 41, resetting it is invalid even when the CPU has already recorded frame 43. Its lifecycle follows the consumer's completion, not the producer's frame number.

Original lab assignment

Create a serial-then-parallel task graph and a checked temporary arena. Pass: resetting live storage is detected in tests, dependency order is verified, and the critical path is identified from a trace.

Assigned study

R11: ownership and concurrency; R20: engine architecture; R26: engine systems as a comparative implementation.

Focused study: R11: ownership and concurrency. R20: engine architecture. S05/S07: extend the ownership proof through GPU completion.

Check your understanding

1. When is resetting an arena safe?
2. Can task C consume A before A has completed its required output?
3. Why implement a serial graph executor first?
4. At offset 17, what range is used by an 8-byte allocation aligned to 16, and how much padding is inserted?
5. For independent A=2 ms, B=5 ms and C=3 ms after both, what are serial time and ideal parallel lower bound?

LESSON 24.4 / MODULE 24

Subsystem boundaries, save formats and reloadability

> **Learning outcome:** Choose interfaces that make change and failure manageable.

Boundaries should reduce the cost of change

Separate world rules from rendering, platform I/O and asset loading. A headless world update should not need a window or a graphics device. The renderer consumes a snapshot or draw description, not unrestricted ownership of every mutable game object. The resource manager resolves asset identities to runtime resources under a declared lifetime policy. The editor requests validated changes through a command interface rather than directly corrupting internal arrays.

An interface should say who owns inputs, how long borrowed data remains valid, which thread may call it and how failure is reported. A function name like load is not enough: does it block, return a ticket, mutate the live world or build a separate candidate? Choosing these semantics is more important than producing a large inheritance hierarchy. [R11]

Saves are a schema, not a memory dump

A durable save stores supported data under an explicit format version. Use stable object IDs, scalar values in declared units and references to named or hashed assets. Do not serialize pointers, container implementation bytes, device handles or accidental padding. Validate types, bounds, finite numbers and referential integrity before committing data to a world.

Migration converts a supported old schema into the current logical model. Unknown future versions should fail clearly unless a deliberate compatibility policy exists. Preserve the original file on a failed migration. A successful parse is not a successful load if it leaves half of the old world replaced by half of the new one.

Load transactionally

Read into bounded temporary storage, validate the complete candidate, resolve its references and then publish it at a controlled boundary. The old world remains usable if any earlier step fails. For saving, write a temporary sibling, flush as required, then replace the destination according to the platform's documented behavior. Atomic replacement is not the same as guaranteed durability after power loss; durability requirements can need additional filesystem and directory synchronization.

A small JSON teaching schema is useful because it is inspectable, but JSON does not remove validation duties. Some parsers accept nonstandard NaN values; reject nonfinite transforms explicitly. Limit file bytes, entity counts, strings and nesting appropriate to the declared threat model. A toy parser is not a sandbox for hostile arbitrary files.

Reload creates overlapping generations

When a shader or texture changes, create a candidate resource while the old resource remains active. Validate the candidate, switch the public handle at a safe boundary and retire the old generation only after its users finish. A failed candidate should leave the old working resource intact. Watch for stale asynchronous completions overwriting a newer successful version; associate requests with a generation or revision token.

Shutdown tests the architecture

Initialize subsystems in an order that matches dependencies and clean up in reverse ownership order. Inject failure after platform initialization, after device creation and after partial asset creation. RAII and scoped ownership help only when wrappers encode the real asynchronous lifetime. A destructor that frees GPU-visible memory too soon is still wrong even though it runs automatically.

The full lab requires one version migration, one malformed-load preservation test, one reloadable asset type and partial-initialization cleanup. The packaged Python schema helper demonstrates validation and migration; it is not a claim that a complete engine's shutdown has already been verified.

Worked example

Version 1 stores an entity's position as x,y,z fields. Version 2 stores position as a three-number array and adds a stable string id. A migration must define where that ID comes from; the teaching fixture uses a known legacy id field and refuses data without it rather than inventing unstable identities.

Load a candidate containing position=[1,2,NaN]. A permissive JSON parser may produce a floating NaN, but schema validation rejects it. The existing world must retain its prior position. Testing only that an exception occurs is insufficient; assert the old world is unchanged.

A texture reload request for revision 5 finishes after revision 6 has already been published. Compare the completion's revision with the current request token. Discard the stale revision-5 result and retire its temporary allocation; never overwrite revision 6 merely because a callback happened later.

Original lab assignment

Add versioned saves and one reloadable asset type. Pass: old supported data migrates, malformed data is rejected without partial world corruption, and subsystem shutdown works after injected initialization failure.

Assigned study

R20: resource management and tools; R26: serialization and gameplay architecture; R11: interfaces and error handling.

Focused study: R11: interfaces and ownership. R20/R26: resources and serialization. Carry Module 20 transactional and failure-testing practices into the engine.

Check your understanding

1. Why is a raw memory pointer unsuitable for a durable save?
2. What must a format version enable?
3. Why test partial initialization failure?
4. Why does atomic file replacement not automatically prove durability after sudden power loss?
5. Revision 5 completes after revision 6 is active. What should the reload system do and why?

Module 24 review

1. What does dropping accumulated ticks sacrifice?
2. Why is a valid handle lookup not permission to hold a component pointer forever?
3. Can an acyclic task graph still race?
4. What makes a load transactional?
5. What protects against an old reload overwriting a new one?

Project handoff

Retain the headless engine core evidence. Explain one failure you deliberately induced and why the fix addresses its cause. Resolve review gaps before treating this module's outputs as reliable inputs to the next one.

Reading references: IDs retain their meanings in source/REFERENCES.md and the full handbook. Use those links alongside the lesson-specific reading assignments.